Bitcoins and Cryptocurrencies

Note: most of the slides used in this course are derived from those available for the book "Bitcoins and Cryptocurrencies Technologies – A Comprehensive Introduction", Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller & Steven Goldfeder, 2016, Princeton University Press.

Simple Local Storage

Simple Local Storage

- It is the simplest way to store bitcoins by putting it into a local device

- To spend a Bitcoin, you need to know:
  - Some **info from the public blockchain**, and
  - The **owner's secret signing key**

So it's all about key management.

How to Store and Use ~~Bitcoins~~ Secret Keys

Goals

Availability: Being able to spend your coins when you want to.

Security: Making sure nobody else can spend your coins.

Convenience: Managing your keys (and thus your coins)

Achieving all the three simultaneously could be a challenge!

Simplest approach - Managing keys

Store key in a file, on your computer or phone

- Very convenient.
- As **available** as your **device**.
  - device lost/wiped ⇒ key lost ⇒ coins lost
- As **secure** as your **device**.
  - device compromised ⇒ key leaked ⇒ coins stolen

Wallet software

- **Software** used **when bitcoins** are **stored locally on a device**
- Wallet software → software that:
  - **Keeps track** of your **coins**.
  - **Manage details** of your **keys**.
  - Provides nice **user interface**.

Nice trick: **use a separate address/key** for **each coin**.
benefits **privacy** (looks like **separate owners**)
wallet can do the **book-keeping**, **user needn't know**

Encoding Address

- **Payments** are made **via exchanging address**.
- The **address** needs to be **encoded or conveyed** to **make a transaction possible.**
- Addresses are exchanged in 2 manners:

    a. Encoded as text string: base58 notation. -> The **key is taken** and encoded in **base58 notation**. Then 58 characters are used to encode digits in base 58 notation. Here likely looking characters and numbers are excluded (O,0);

    a. QR (Quick Response) code -> 2D barcode, Phone can scan the 2D barcode and extract the bits of the address.

Encoding addresses

Encode as **text string**: base58 notation

```
123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz
```

or use **QR code**

Base58

Representing values with an easy-to-share set of characters.

```
  1 2 3 4 5 6 7 8 9
A B C D E F G H   J K L M N   P Q R S T U V W X Y Z
a b c d e f g h i j k   m n o p q r s t u v w x y z
```

Vanity addresses

- A vanity address is a **public crypto address** with **unique letters and digits** that are often **chosen by the owner**.

- For example, the gambling website Satoshi Bones has users send money to **addresses containing the string "bones" in positions 2--6**, such as

  1bonesEeTcABPjLzAb1VkFgySY6Zqu3sX

  (all **regular addresses** begin with the **character 1**, indicating **pay-to-pubkey-hash.**)

Vanity addresses

- We said that addresses are outputs of a hash function, which produces random-looking data, so how did the string “bones” get in there?

- If Satoshi Bones were simply making up these addresses, lacking the ability to invert hash function, they wouldn’t know the corresponding private keys and hence wouldn’t actually control those addresses.

- Instead, they repeatedly generated private keys until they got lucky and found one which hashed to this pattern. Such addresses are called ***vanity addresses*** and there are tools to generate them.

Vanity addresses - How much work does this take?

- Since there are **58 possibilities for every character**, if you want to **find an address** which starts with a **specific k -character string**, you'll need to **generate 58 k addresses** on **average until you get lucky**.

- So **finding an address** starting with **"bones"** would have required **generating over 600 million addresses**! This can be done on a **normal laptop nowadays**.

- But **it gets exponentially harder with each extra character**.

- **Finding a 15-character prefix** would **require an infeasible amount of computation** and (**without finding a break in the underlying hash function**) should be **impossible**.

Hot and Cold Storage

Hot and Cold Storage

- Hot storage -> Online (Convenient but risky) - (money in your wallet);
- Cold storage -> Offline (Archival but safer) - (money in the safe);
- How to moved between hot and cold sides (relationship)?
  - **Each side** will have to have **separate secret keys**;
  - **Each side** should know the **others address** in order to **make payments.**

Hot storage

online

convenient but risky

Cold storage

offline

archival but safer

separate keys

Hot storage

online

hot secret key(s)

cold address(es)

payments

Cold storage

offline

cold secret key(s)

hot address(es)

Hot and Cold Storage

- In practice, **hot storage is online** and **cold storage is offline** and hence they cannot connect to each other.
- But the **hot storage knows the address** at **which the cold storage accepts payment**, therefore the **hot storage can send coins across** to the **cold storage**, even **while cold storage is offline**.
- Next time **the cold storage connects** it will **receive the information from a block-chain** about those **transfers**.

Hot storage

online

hot secret key(s)

cold address(es)

payments

Cold storage

offline

Problem:

Want to **use a new address (and key)** for **each coin sent to cold**
But how can hot wallet learn new addresses if cold wallet is offline?

Awkward solution:

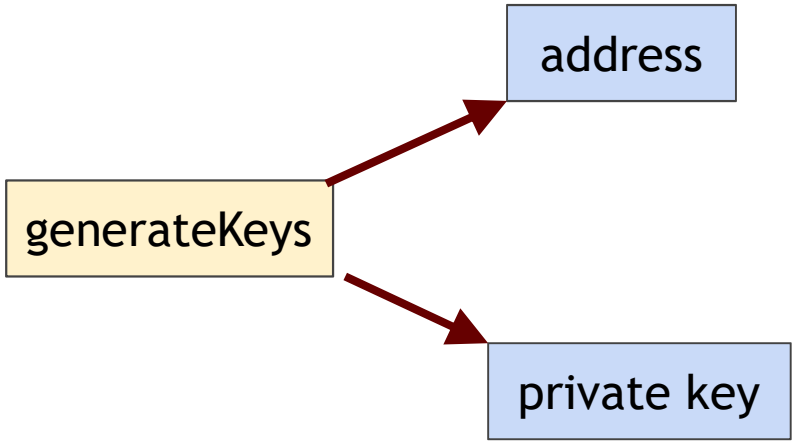
- **Generate** a **big batch of addresses/keys**, **transfer to hot beforehand**.

- In case the **addresses run out then reconnect** the **hot-side to the cold-side** in order to **fetch more addresses**.

Better solution:

Hierarchical wallet. It requires cryptographic trickery.

Regular key generation:



Hierarchical wallets

- It allows the **cold side** to use an essentially **unbounded number of addresses** and the **hot side to know about these addresses,** but with **only a short, one-time communication between the two sides**.

- But it requires a little bit of cryptographic trickery.

- Instead of generating a single address **we generate** what we'll call **address generation info**, and rather than a private key we generate what we'll call **private key generation info**.

Hierarchical wallets

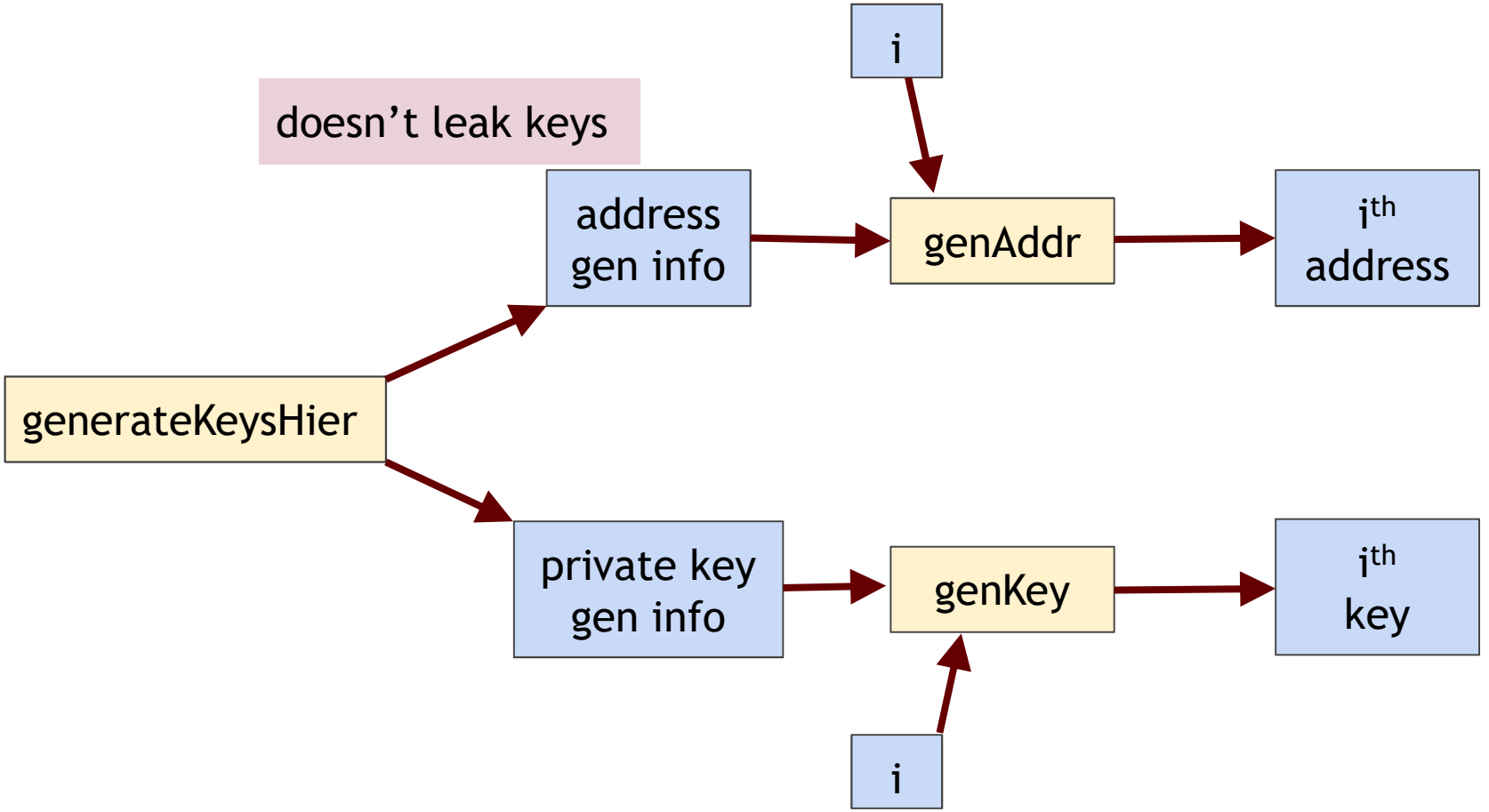
- Given the **address generation info**, we can **generate a sequence of addresses**:
  - We apply an **address generation function** that takes **as input the address generation info and any integer i and generates the i'th address** in the sequence.

- Similarly we can generate a **sequence of private keys** using **the private key generation info.**

Hierarchical wallets

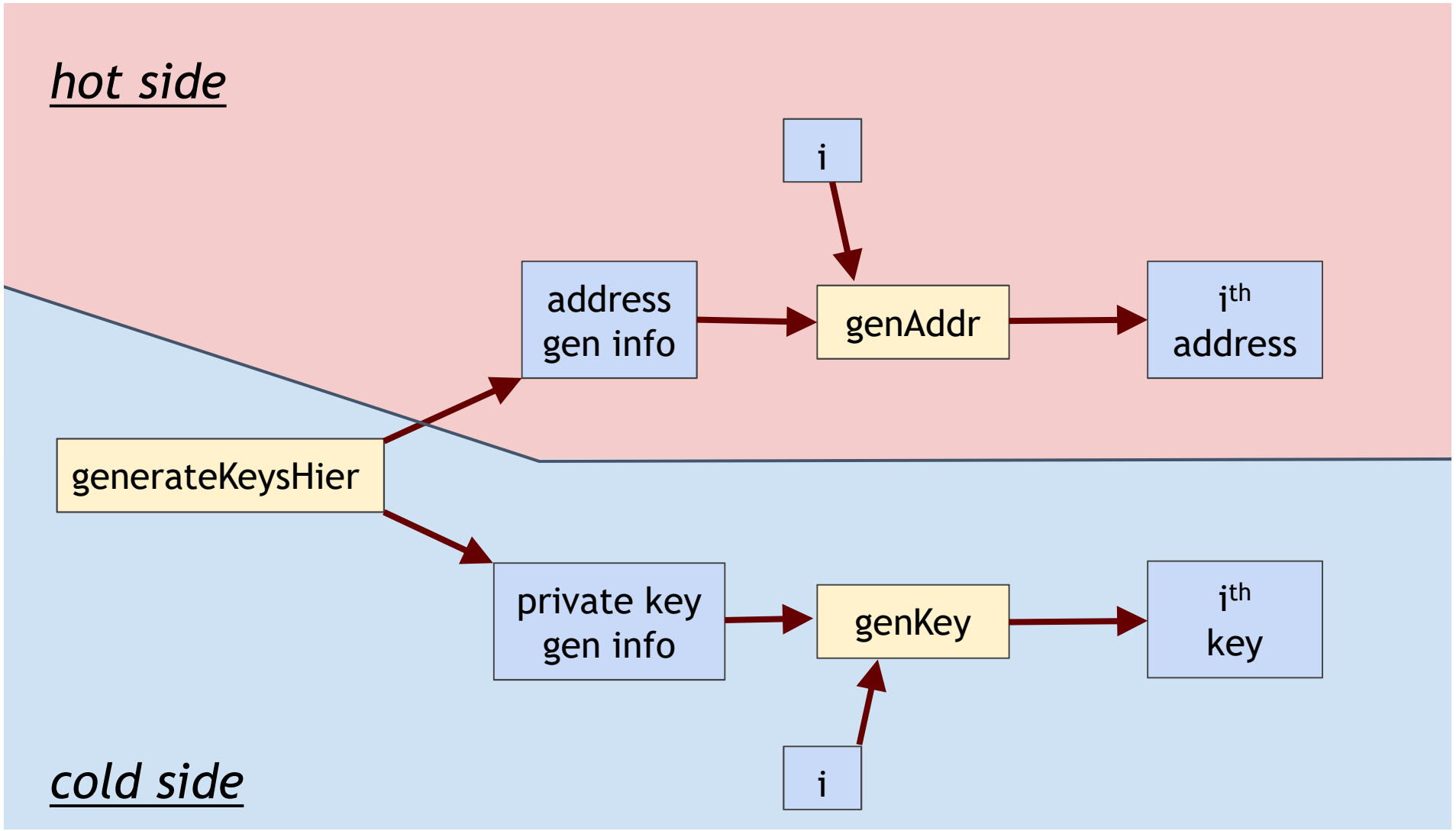
- **Properties:**
  - **The ith address and the ith Key match up and correspond to each other.**
  - The address generation info **doesn't leak any information about the private keys. That means that it's safe to give the address generation info to anybody.**
  - **Public keys (addresses) won't be linkable to each other - it won't be possible to infer that they come from the same wallet.**
- **Not all digital signature schemes** that exist can be modified to support **hierarchical key generation**.
- Bitcoin, **ECDSA, does support hierarchical key generation**.

Hierarchical key generation:



Hierarchical wallets

- That is, the **cold side generates arbitrarily many keys** and **the hot side generates the corresponding addresses**.
- The cold side creates and saves private key generation info and address generation info.
- It does a one-time transfer of the latter to the hot side.
- The hot side generates a new address sequentially every time it wants to send coins to the cold side.
- **When the cold side reconnects, it generates addresses sequentially and checks the block chain for transfers to those addresses until it reaches an address that hasn't received any coins.**
- **Cold side** can also generate private keys sequentially if it wants to send some coins back to the hot side or spend them some other way.


hot side
i
address
gen info
genAddr
ith
address
generateKeysHier
private key
gen info
genKey
ith
key
i
cold side

Hierarchical wallets

- Here we have two levels of security, with the hot side being at a lower level. If the hot side is compromised, the unlinkability property that we just discussed will be lost, but the private keys (and the bitcoins) are still safe.

- This scheme supports arbitrarily many security levels --- hence “hierarchical”.

- Cold informationwhether one or more keys, or key-generation info — can be stored. The first way is to store it in some kind of device and put that device in a safe.

- It might be a laptop computer, a mobile phone or tablet, or a thumb drive. The important thing is to turn the device off and lock it up, so that if somebody wants to steal it they have to break into the locked storage.

How to store cold info

(1) Info stored in device, device locked in a safe
(2) “Brain wallet”
encrypt info under passphrase that user remembers
(3) Paper wallet
print info on paper, lock up the paper
(4) In “tamperproof” device
device will sign things for you, but won’t divulge keys

In general, a combination of one more more of these techniques can be used

Splitting and Sharing Keys

Current key storage schemes

- **Storing the Key at a certain place alone - could cause a single point of failure problem.**
- **In order to avoid this situation, the key needs to be shared or split.**

- **Problem**: single point of failure

- **Trivial solution**: make multiple copies or backup
  - **Advantage**: Availability improves
  - **Disadvantage**: Security of stored keys is worst (multiple avenues to steal)

- **Question**: Can we improve both availability and security?
- **Answer**: Surprisingly, yes! Using some cute cryptographic / mathematical tricks → Secret Sharing!

Secret sharing

- We want to **divide our secret key** into some **number N of pieces.**
- We want to do it in such a way that if **we're given any K** of those pieces then we'll be able to reconstruct the original secret.
- However, if we're **given fewer than K pieces** then we won't be able to learn anything about the original secret.

Secret sharing

- Let’s say we have N=2 and K=2. That means we're generating 2 shares based on the secret.
- We need both shares to be able to reconstruct the secret. Let’s call our secret S, which is just a big (say 128-bit) number.
- We could generate a 128-bit random number R and make the two shares be R and S $\oplus$ R. ($\oplus$ represents bitwise XOR).
- Essentially, we’ve “encrypted” S with a one-time pad, and we store the key (R) and the ciphertext (S $\oplus$ R) in separate places.
- Neither the key (R) nor the ciphertext by itself tells us anything about the secret.
- But given the two shares, we simply XOR them together to reconstruct the secret.

Secret sharing

- **This trick works as long as N and K are the same** — we'd just need to *generate N-1 different random numbers* for the *first N-1 shares*.
- And the final share would be the secret XOR'd with all other N-1 shares.
- **But if N is more than K, this doesn't work any more, and we need some algebra.**

Secret sharing

Idea: split secret into N pieces, such that
given any K pieces, can reconstruct the secret
given fewer than K pieces, don't learn anything

Example: N=2, K=2
P = a large prime
S = secret in [0, P)
R = random in [0, P)

split:
$X_1 = (S+R) \bmod P$ $X_2 = (S+2R) \bmod P$

reconstruct:
$(2X_1-X_2) \bmod P = S$

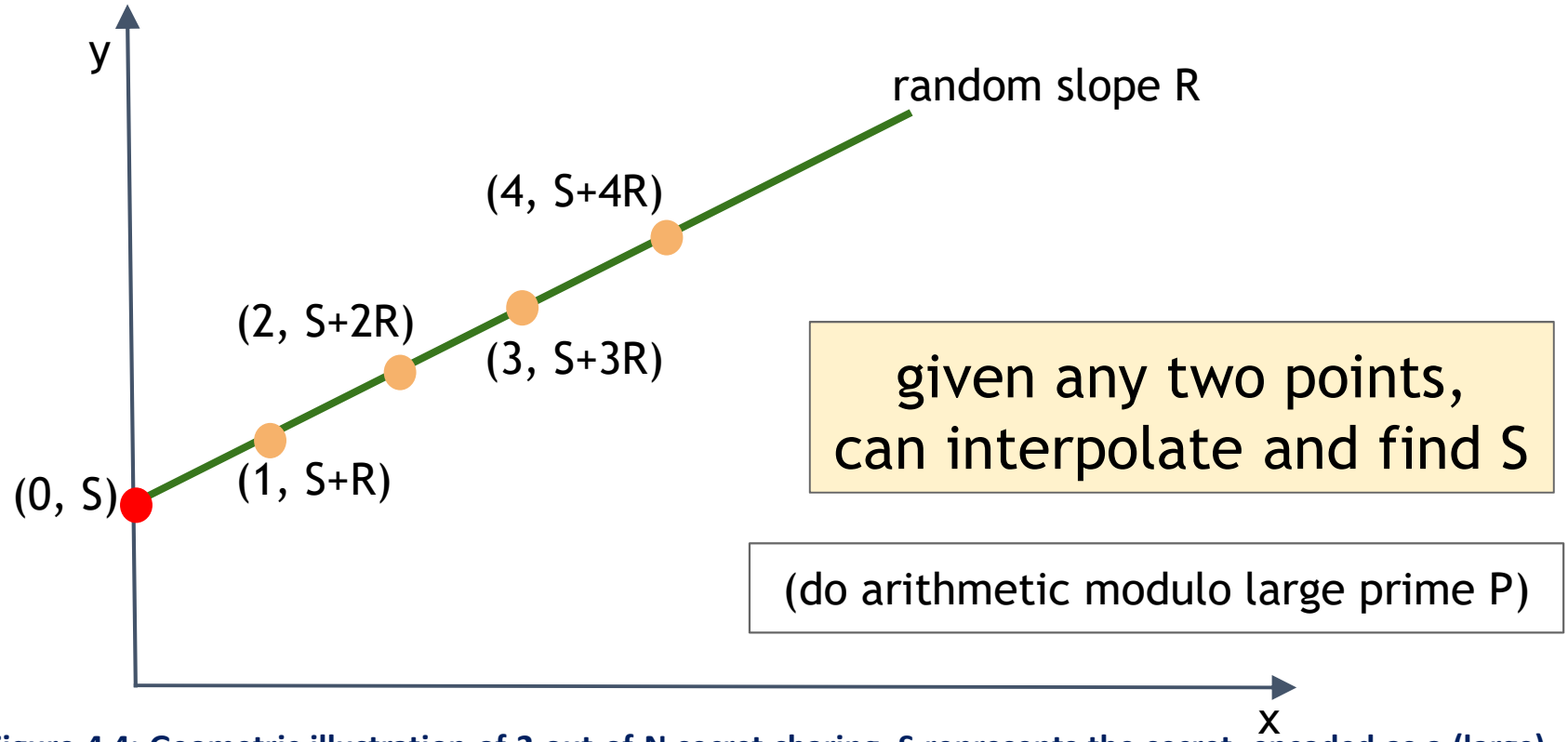


Figure 4.4: Geometric illustration of 2-out-of-N secret sharing. S represents the secret, encoded as a (large) integer. The green line has a slope chosen at random. The orange points (specifically, their Y-coordinates S+R, S+2R, ...) correspond to shares. Any two orange points are sufficient to reconstruct the red point, and hence the secret. All arithmetic is done modulo a large prime number.

Secret sharing

- Take a look at Figure 4.4. What we've done here is to first generate the point (0, S) on the Y-axis,
- And then drawn a line with a random slope through that point.
- Next we generate a bunch points on that line, as many as we want.
- It turns out that this is a **secret sharing of S with N being the number of points we generated and K=2.**

Secret sharing

- There's only one other subtlety: to make the math work out, we have to do all our arithmetic modulo a large prime number P.
- It doesn't need to be secret or anything, just really big.
- And the **secret S has to be between 0 and P-1, inclusive**.
- So when we say we generate points on the line, what we mean is that we generate a random value R, also between 0 and P-1, and the points we generate are
  - x=1, y=(S+R) mod P
  - x=2, y=(S+2R) mod P
  - x=3, y=(S+3R) mod P
  - and so on.
- The secret corresponds to the point x=0, y=(S+0*R) mod P, which is just x=0, y=S.

Secret sharing

- What we've seen is a way to do secret sharing with K=2 and any value of N.
- This is already pretty good — if N=4, say, you can divide your secret key into 4 shares and put them on 4 different devices so that if someone steals any one of those devices, they learn nothing about your key.
- On the other hand, even if two of those devices are destroyed in a fire, you can reconstruct the key using the other two.
- So as promised, **we've increased both availability and security.**
- We can do **secret sharing with any N and K as long as K is no more than N.**

Secret sharing

Table 4.1: The math behind secret sharing. Representing a secret via a series of points on a random polynomial curve of degree K-1 allows the secret to be reconstructed if, and only if, at least K of the points ("shares") are available.

Equation	Random parameters	Points needed to recover S
$(S + RX) \bmod P$	R	2
$(S + R_1X + R_2X^2) \bmod P$	R_1, R_2	3
$(S + R_1X + R_2X^2 + R_3X^3) \bmod P$	R_1, R_2, R_3	4

etc.

support K-out-of-N splitting, for any K, N

Equation	Degree	Shape	Random parameters	Number of points (K) needed to recover S
$(S + RX) \bmod P$	1	Line	R	2
$(S + R_1X + R_2X^2) \bmod P$	2	Parabola	R_1, R_2	3
$(S + R_1X + R_2X^2 + R_3X^3) \bmod P$	3	Cubic	R_1, R_2, R_3	4

Table 4.1: The math behind secret sharing. Representing a secret via a series of points on a random polynomial curve of degree K-1 allows the secret to be reconstructed if, and only if, at least K of the points ("shares") are available.

Secret sharing

- We're **safe** even if an **adversary learns up to K-1 of them.**
- And at the same time we can **tolerate the loss of up to N-K of them**.
- The online wallet provider will need some way to avoid a single point of failure when storing their keys.

Secret sharing

Good: Store shares separately, adversary must compromise several shares to get the key.

Bad: To sign, need to bring shares together, reconstruct the key. ⇐ vulnerable

(**The point where we bring all the shares together is still a single point of vulnerability where an adversary might be able to steal the key** - Threshold cryptography can be used to solve this problem).

Threshold cryptography

- If the shares are stored in different devices, there's a way to produce Bitcoin signatures in a decentralized fashion without ever reconstructing the private key on any single device. This is called a "threshold signature."

- The best use-case is a wallet with two-factor security, which corresponds to the case N=2 and K=2.

Threshold cryptography

- Say you've configured your wallet to split its key material between your desktop and your phone.
- Then you might initiate a payment on your desktop, which would create a partial signature and send it to your phone.
- Your phone would then alert you with the payment details — recipient, amount, etc. — and request your confirmation.
- If the details check out, you'd confirm, and your phone would complete the signature using its share of the private key and broadcast the transaction to the block chain.
- If there were malware on your desktop that tried to steal your bitcoins, it might initiate a transaction that sent the funds to the hacker's address, but then you'd get an alert on your phone for a transaction you didn't authorize, and you'd know something was up.

Multi-sig

- There's an entirely different option for avoiding a single point of failure: multi-signatures

Lets you keep shares apart, approve transaction without reconstructing key at any point.

Example

Andrew, Arvind, Ed, and Joseph are co-workers.
Their company has lots of Bitcoins.

Each of the four generates a key-pair,
puts secret key in a safe, private, offline place.

The company's cold-stored coins use multi-sig, so that three of the four keys must sign to release a coin.

Online Wallets and Exchanges

Online Wallets

- An online wallet is kind of like a local wallet that you might **manage yourself**, except the **information is stored in the cloud**, and you **access** it using a **web interface on your computer** or **using an app on your smartphone.**

- Some **online wallet services** that are popular in early 2015 are **Coinbase and blockchain.info.**

Online Wallets

- The point of view of security is that **the site delivers the code that runs on your browser or the app**, and it **also stores your keys**.

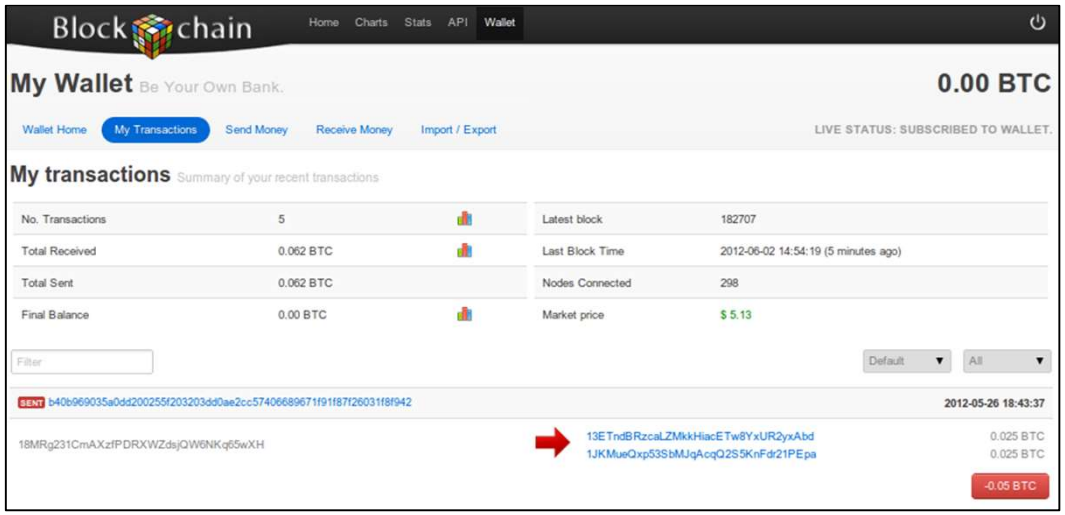
- The **site** will **encrypt those keys under a password** that only you **know,** but of course **you have to trust them to do that**.

- You have to **trust their code** to **not leak your keys or your password**

Online wallet

- Like a local wallet
  - but “in the cloud”

- Runs in your browser
- Site sends code
- Site stores keys
- You log in to access wallet



Online wallet tradeoffs

Convenient: **nothing to install** on your **computer**, works on **multiple devices using a browser (the real wallet lives in the cloud).**

- On your **phone you maybe just have to install an app** once, and **it won't need to download the block chain**.

Security worries: vulnerable if **site is malicious or compromised** - bitcoins are in trouble.

Ideally, site is run by security professionals - you have to trust them and you have to rely on them not being compromised.

Bitcoin exchanges - Bank-like services

- You give the bank money (a “deposit”)
- Bank promises to pay you back later, on demand

- Bank doesn’t actually keep your money in the back room
  - Typically, bank invests the money
  - Keeps some around to meet withdrawals (“**fractional reserve**”) - **fractional reserve where they keep a certain fraction of all the demand deposits on reserve just in case.**

Bitcoin Exchanges

- Accept deposits of Bitcoins and fiat currency ($, €, …)
  - Promise to pay back on demand

- Lets customers:
  - Make and receive Bitcoin payments
  - Buy/sell Bitcoins for fiat currency
    - typically, match up BTC buyer with BTC seller - **Typically they do this by finding some customer who wants to buy bitcoins with dollars and some other customer who wants to sell bitcoins for dollars, and match them up**.

What happens when you buy BTC

- Suppose my account at Exchange holds $5000 + 3 BTC and I use Exchange to buy 2 BTC for $580 each
  - **Result**: my account holds $3840 + 5 BTC

- **Note**: **no BTC transaction appears on the blockchain**
  Only effect: **Exchange is making a different promise now:**

Before they said, “we'll give you 5000 USD and 3 BTC” and now they're saying “we'll give you 3840 USD and 5 BTC.”
It's just a change in their promise — no
actual movement of money through the dollar economy or through the block chain.

Exchanges: Pros and Cons

- **Pros**:
  1. Connects BTC economy to fiat currency economy
  2. Easy to transfer value back and forth

- **Cons**:
  1. Risk - same kinds of risks as banks
     i. Three types of risks: Bank run, Ponzi scheme, hack

- A study in 2013 found that 18 of 40 Bitcoin exchanges had ended up closing due to some failure or some inability to pay out the money that the exchange had promised to pay out.

Risk 1: Bank Run

The first risk is the risk of a ***bank run*** . A run is what happens when a **bunch of people show up all at once and want their money back**.

Risk 2: “Ponzi” Schemes or Cheating

Charles Ponzi

This is a scheme where someone gets people to give them money in exchange for profits in the future, but then actually takes their money and uses it to pay out the profits to people who bought previously.

Risk 3: Security Threats (Cyber and or Physical)

The third risk is that of a hack, the risk that someone — **perhaps even an employee of the exchange** — will manage to **penetrate the security of the exchange.**

Study: 45 percent of Bitcoin exchanges end up closing

TECHNOLOGY / 26 APRIL 13 / by IAN STEADMAN

A study of the Bitcoin exchange industry has found that 45 percent of exchanges fail, taking their users' money with them. Those that survive are the ones that handle the most traffic -- but they are also the exchanges that suffer the greatest number of cyber attacks.

Computer scientists Tyler Moore (from the Southern Methodist University, Dallas) and Nicolas Christin (of Carnegie Mellon University) found 40 exchanges on the web which offered a service of changing bitcoins into other fiat currencies or back again. Of those 40, 18 have gone out of business -- 13 closing without warning, and five closing after suffering security breaches that forced them to close. Four other exchanges have

Almost half of all exchanges close *Shutterstock*


東京でMT.GOXのデモ
へ参加してください。
東京都渋谷区渋谷
2丁目11-5
MTGOX
WHERE IS
OUR MONEY

Bank Regulation

- For traditional banks, government typically:
  - Imposes minimum reserve requirements
    - **Must hold some fraction of deposits in reserve**
  - Regulates behavior, investments
  - Insures depositors against losses
  - Acts as lender of last resort

  - Reserve requirement in the U.S. is typically 3-10%.
  - Bitcoin exchanges are not regulated.

Proof of Reserve

- Bitcoin exchange can prove it has fractional reserve.
  - Fraction can be 25% or 100%
- Prove how much reserve you're holding:
  - Publish valid payment-to-self of that amount
  - Sign a challenge string with the same private key
- Prove how many demand deposits you hold: ...

Proof of Reserve

- The first is to prove how much reserve you're holding — that's the relatively easy part.
- The company simply publishes a valid payment-to-self transaction of the claimed reserve amount.
- That is, if they claim to have 100,000 bitcoins, they create a transaction in which they pay 100,000 bitcoins to themselves and show that that transaction is valid.
- **Then they sign a challenge string — a random string of bits generated by some impartial party** — with the **same private key** that was used to **sign the payment-to-self transaction**

Proof of Reserve

- This proves that someone who knew that private key participated in the proof of reserve.
- Also, note that you could always under-claim: the organization might have 150,000 bitcoins but choose to make a payment-to-self of only 100,000.
- So this proof of reserve doesn't prove that this is all you have, but it proves that you have at least that much.

Proof-of-Reserve Problem

Bitcoin exchanges can prove a lower bound on fractional reserve by providing:

1. Lower bound for reserves

2. Upper bound for liabilities

Proof of Reserve

Q: How to **prove** how much reserve you are holding?

1. Publish a valid **payment-to-self** of claimed amount.
2. Sign **challenge string** with same private key.

Now the hard part . . .

Proof of Liabilities

- The second piece is to prove **how many demand deposits you hold, which is the hard part**.
- If you can **prove your reserves and your demand deposits** then anyone can **simply divide those two numbers** and **that's what your fractional reserve is**.
- We'll present a **scheme** that **allows** you to ***over-claim*** **but not under-claim your demand deposits.**
- So **if you can prove** that your **reserves are at least a certain amount** and **your liabilities are at most a certain amount**, **taken together, you've proved a lower bound on your fractional reserve.**

Proof of Liabilities

Vanilla approach:

Publish list of amounts and usernames of all accounts!

Users can complain if their accounts are missing or amounts are wrong.

Exchange can create fake users, but this only overstates liabilities.

Problem: What about customer privacy?!!

Proof of Liabilities

- If you **didn't care at all about the privacy of your users**, you could simply publish your records — specifically, **the username and amount of every customer with a demand deposit.**

- Now **anyone can calculate your total liabilities**, and if **you omitted any customer or lied about the value of their deposit**, you **run the risk that that customer will expose you.**

- You **could make up fake users**, but you **can only increase the value of your claimed total liabilities this way**.

- So as long as there **aren't customer complaints**, this lets **you prove a lower bound on your deposits**.

- The trick, of course, is to do all this while respecting the privacy of your users.

Proof of Liabilities

- Recall that a **Merkle tree is a binary tree** that's **built with hash pointers** so that **each of the pointers** not only says **where we can get a piece of information**, but also **what the cryptographic hash of that information** is.

- The **exchange executes the proof** by **constructing a Merkle tree** in which **each leaf corresponds to a user**, and **publishing its root hash**.

- Similar to the naive protocol above, **it's each user's responsibility to ensure that they are included in the tree**.

- In addition, there's a way for users to collectively check the claimed total of deposits.

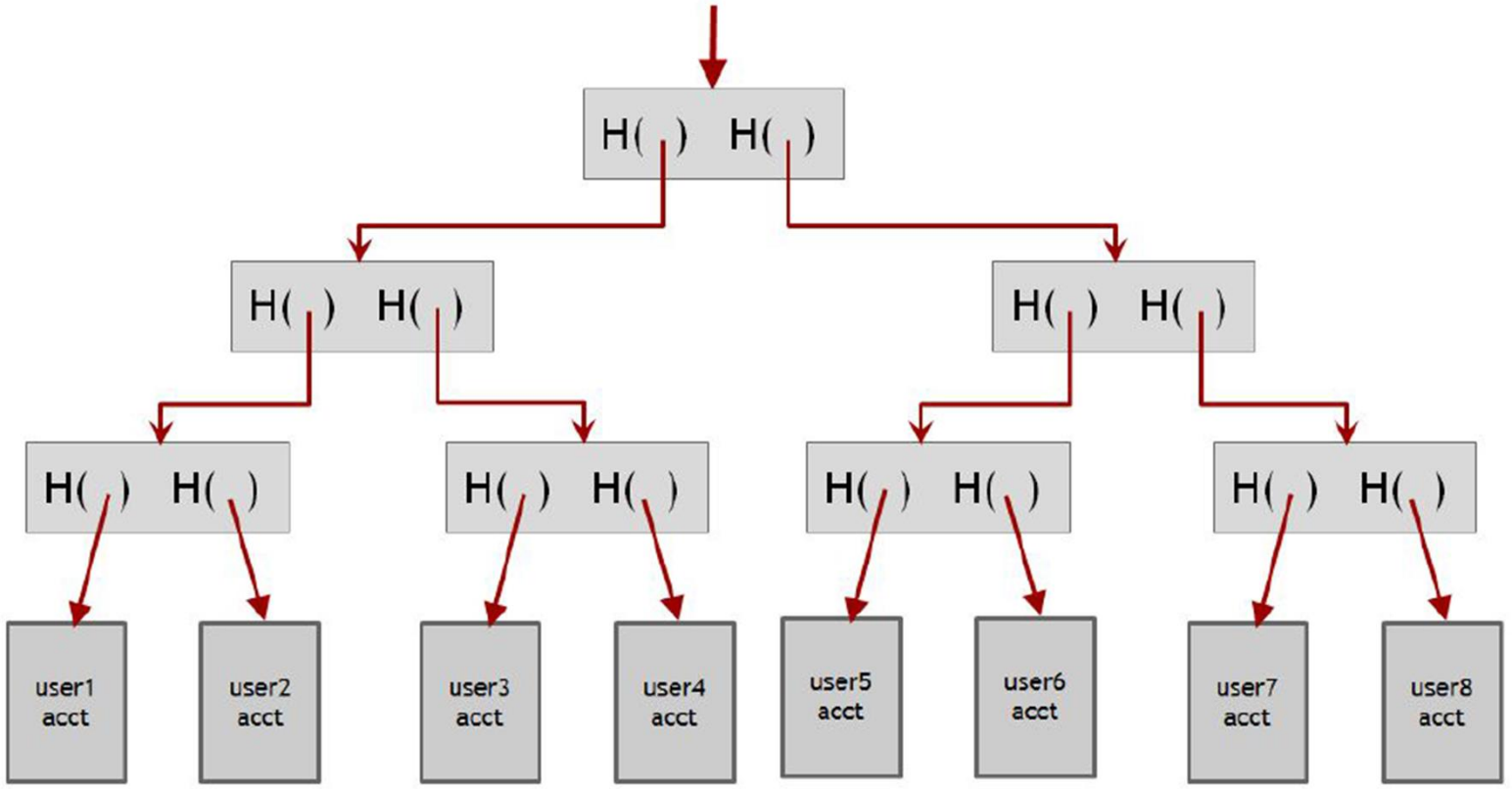
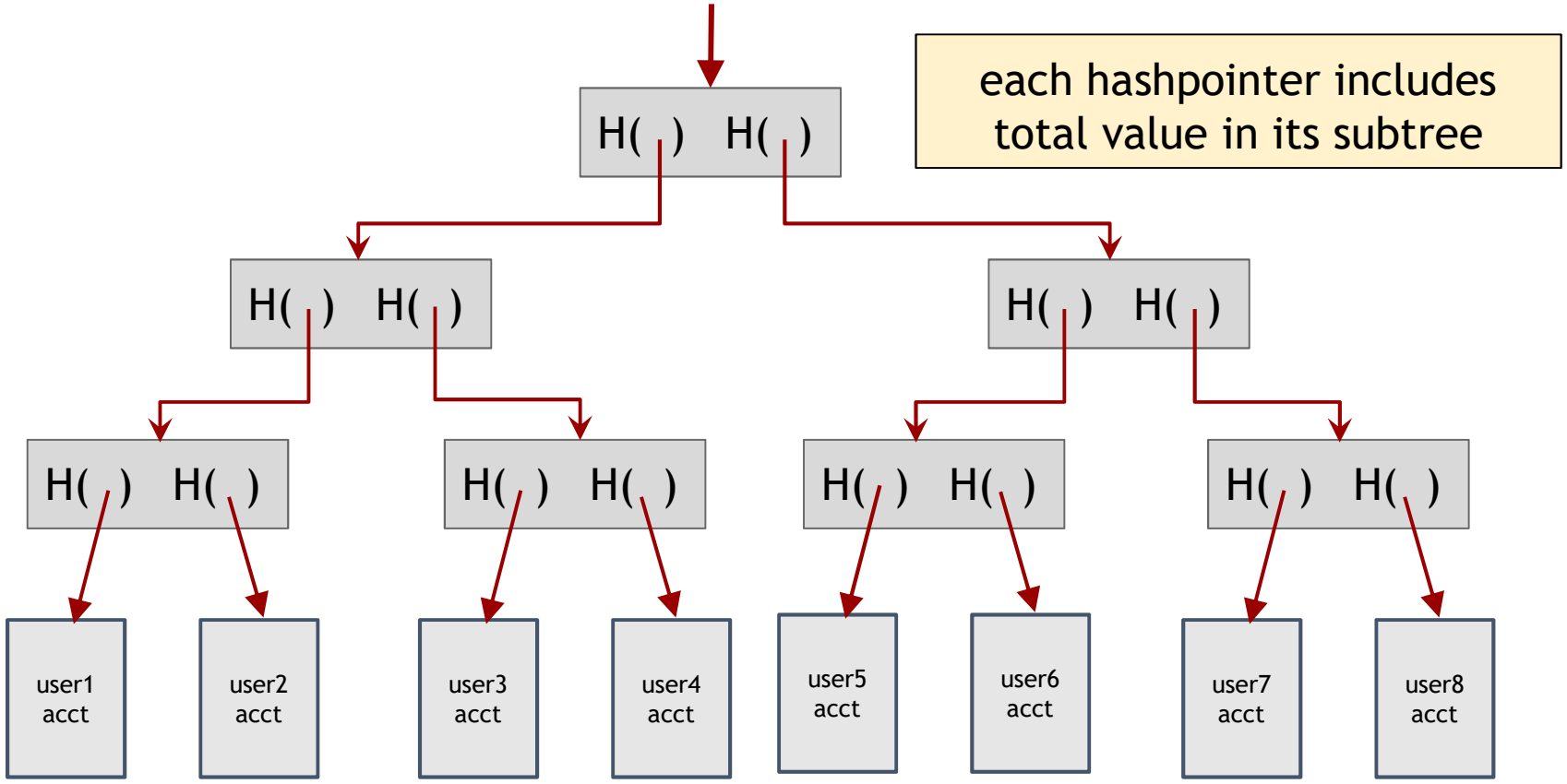


Figure 4.5: Proof of liabilities. The exchange publishes the root of a Merkle tree that contains all users at the leaves, including deposit amounts. Any user can request a proof of inclusion in the tree, and verify that the deposit sums are propagated correctly to the root of the tree.

Merkle tree with subtree totals



Proof of Liabilities

- The **exchange constructs this tree**, **cryptographically signs the root pointer along with the root attribute value**, and **publishes it**.

- The **root value is of course the total liabilities**, the **number we're interested in**.

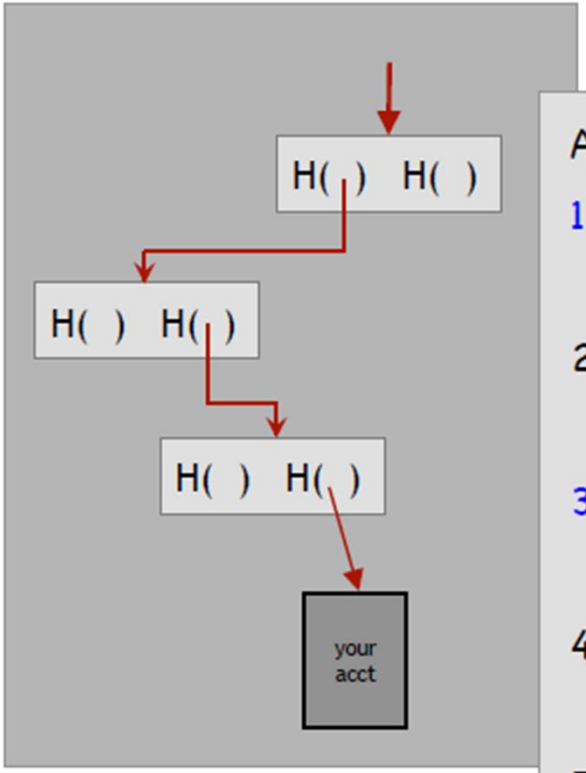
- The **exchange is making the claim that all users are represented in the leaves of the tree**, their deposit values are represented correctly, and that the values are propagated correctly up the tree so that the **root value is the sum of all users' deposit amounts**.

Proof of Liabilities

- Now **each customer can go to the organization** and ask for a **proof of correct inclusion.**

- The **exchange must then show the customer the partial tree from that user's leaf up to the root,** as shown in Figure 4.6.

Are you in the Tree?



As customer you can verify that:

1. Root hash pointer and root value are what exchange published.
2. Hash pointers are consistent all the way down.
3. Leaf contains correct information (customer no. and amount)
4. Each value is sum of the values of subtrees beneath it.
5. Neither of values is negative number.

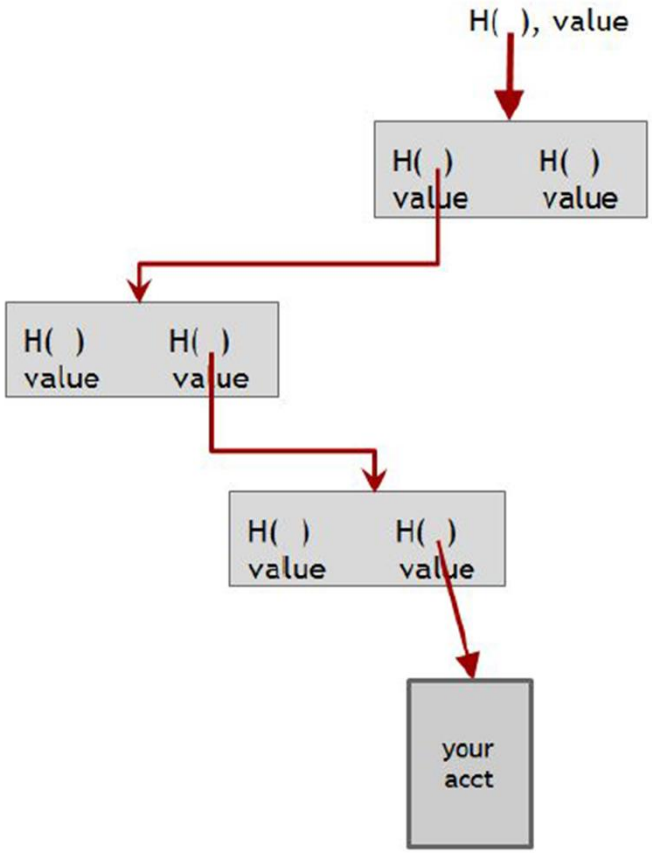
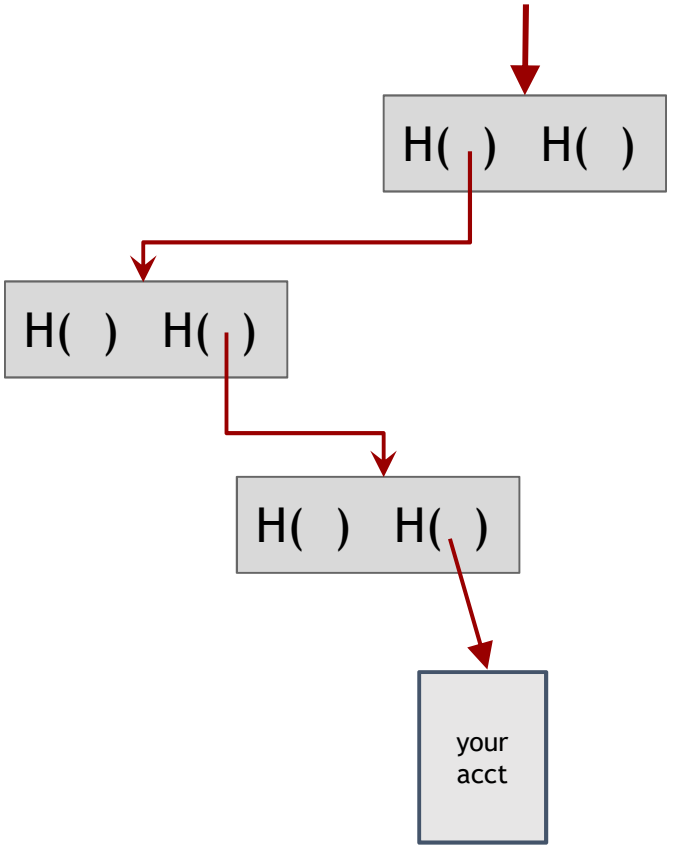


Figure 4.6: Proof of inclusion in a Merkle tree. The leaf node is revealed, as well as the siblings of the nodes on the path from the leaf to the root.

Checking that you're represented in the tree



show O(log n) items

Proof of Liabilities

- Crucially, the exchange cannot present different values in any part of the tree to different customers.

- That's because doing so would either imply the ability find a hash collision, or presenting different root values to different customers, which we assume is impossible.

Proof of Reserve

- Prove that you have at least X amount of reserve currency
- Prove that customers have at most Y amount deposited
- So reserve fraction ≥ X / Y

Exchange

- You might notice that the two proofs presented here (the proof of reserves by signing a challenge string and the proof of liabilities via a Merkle tree) reveal a lot of private information.
- Specifically, they reveal **all of the addresses being used by the exchange**, the **total value of the reserves** and **liabilities, and even some information about the individual customers balances**.
- **Real exchanges are hesitant to publish this**, and as a **result cryptographic proofs of reserve have been rare.**
- **A recently proposed protocol called Provisions enables the same proof-of-solvency, but without revealing the total liabilities or reserves or the addresses in use.**

Payment Services

Scenario: merchant accepts BTC

- **Customer wants**: to pay with Bitcoin

- **Merchant wants**:
  - To receive dollars
  - Simple deployment
  - Low risk (tech risk, security risk, exchange rate risk)

Payment Services

- There are various possible risks: using new technology may cause their website to go down, costing them money.
- There's the security risk of handling bitcoins — someone might break into their hot wallet or some employee will make off with their bitcoins.
- Finally there's the exchange rate risk: the value of bitcoins in dollars might fluctuate from time to time.
- The merchant who might want to sell a pizza for twelve dollars wants to know that they're going to get twelve dollars or something close to it, and that the value of the bitcoins that they receive in exchange for that pizza won't drop drastically before they can exchange those bitcoins for dollars.

Payment Services

- Payment services exist to allow both the customer and the merchant to get what they want, bridging the gap between these different desires.

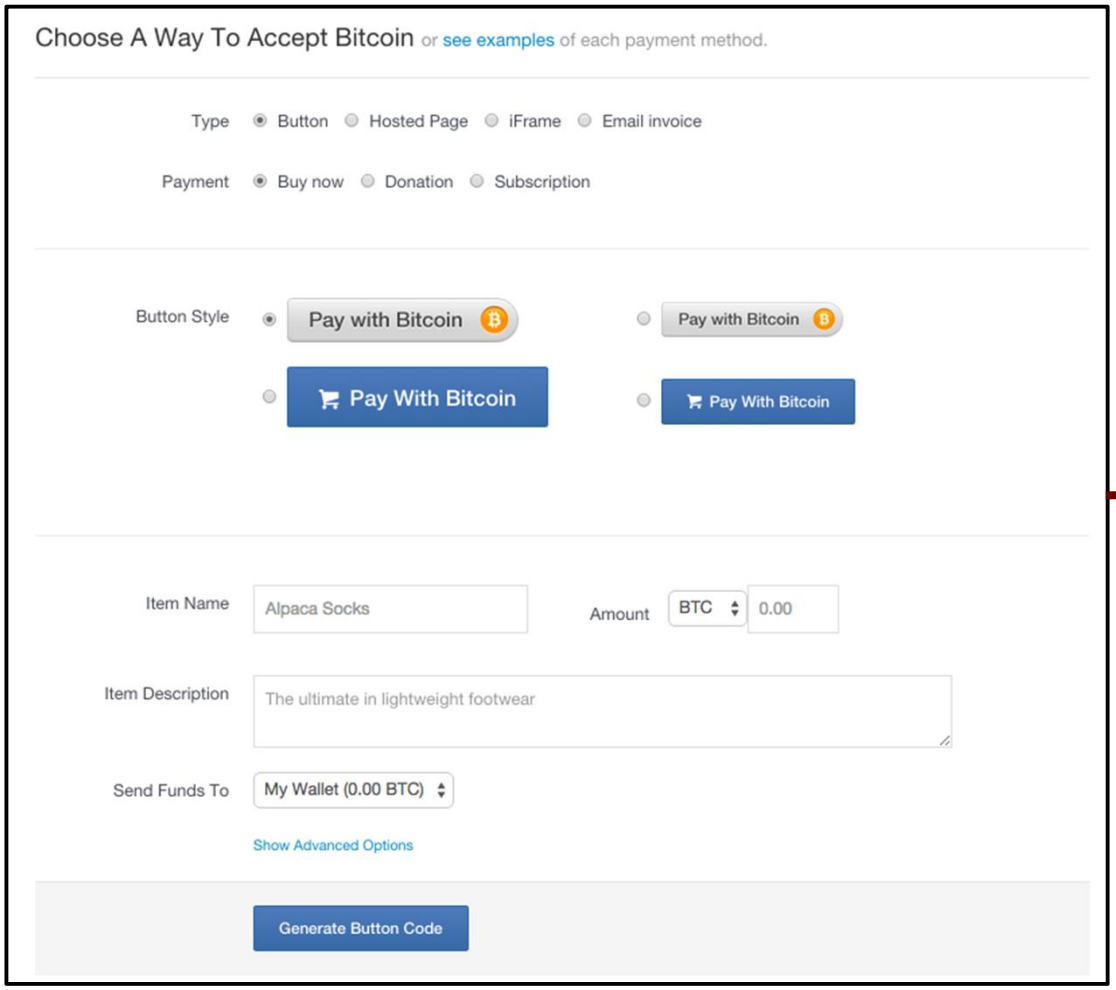


Figure 4.7: Example payment service interface for generating a pay-with-Bitcoin button. A merchant can use this interface to generate a HTML snippet to embed on their website.

HTML for payment button

Payment Services

- The process of receiving Bitcoin payments through a payment service might look like this to the merchant:
  - 1. The merchant goes to payment service website and fills out a form describing the item, price, and presentation of the payment widget, and so on. Figure 4.7 shows an illustrative example of a form from Coinbase.
  - 2. The payment service generates HTML code that the merchant can drop into their website.
  - 3. When the customer clicks the payment button, various things happen in the background and eventually the merchant gets a confirmation saying, "**a payment was made by customer ID [customer-id] for item [item-id] in amount [value].**"

Payment Services

- While this manual process makes sense for a small site selling one or two items, or a site wishing to receive donations, copy-pasting HTML code for thousands of items is of course infeasible.

- So payment services also provide programmatic interfaces for adding a payment button to **dynamically generated web pages.**

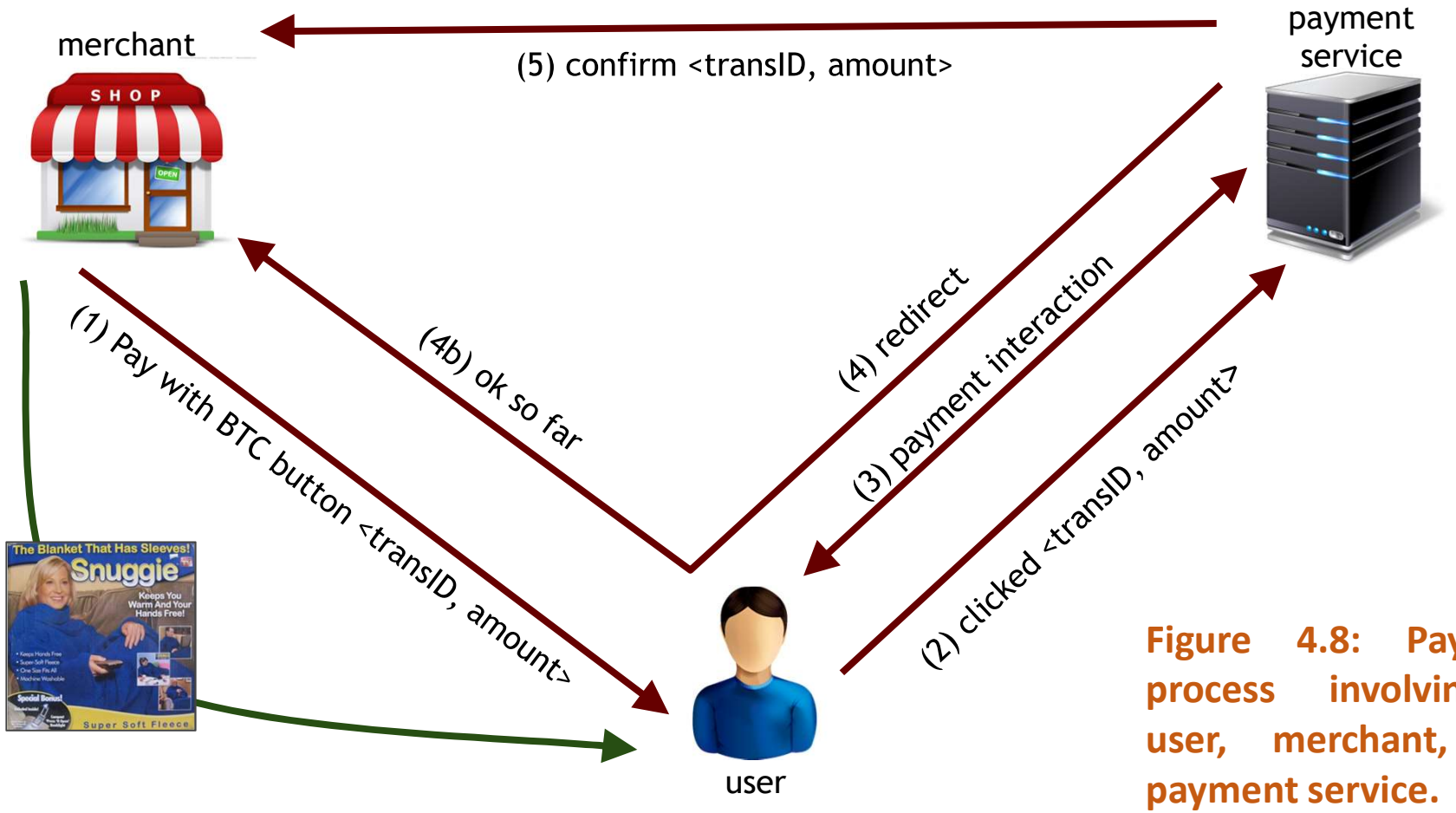


Figure 4.8: Payment process involving a user, merchant, and payment service.

End result

customer: pays Bitcoins

merchant: gets dollars, minus a small percentage

payment service:
- gets Bitcoins
- pays dollars (keeps small percentage)
- **absorbs risk: security, exchange rate**
- needs to exchange Bitcoins for dollars, in volume

Transaction Fees

Transaction Fees

- **Recall**:
  - Transaction fee = value of inputs - value of outputs
  - Fee goes to miner who records the transaction

- How are transaction fees set today?

Transaction Fees

- Why do transaction fees exist at all? The reason is that there is some cost that **someone has to incur in order to relay your transaction.**

- The **Bitcoin nodes need to relay your transaction** and **ultimately a miner needs to build your transaction into a block**, and it costs them a little bit to do that.

- For example, if a miner's block is slightly larger because it contains your transaction, it will take slightly longer to propagate to the rest of the network and **there's a slightly higher chance that the block will be orphaned if another block was found near-simultaneously by another miner.**

Transaction Fees

- It costs resources for
  - Peers to relay your transaction
  - Miner to record your transaction
  - Transaction fee compensates for (some of) these costs

- Generally, higher fee means transaction will be forwarded and recorded faster!

Transaction Fees

- Nodes **don’t receive monetary compensation in the current system**, although running a node is of course far less expensive than being a miner.

- Generally you're free to set the transaction fee to whatever you want it to be.

- You can pay no fee, or if you like you can set the fee quite high.

- As a general matter, if you pay a higher transaction fee it's natural that your transaction will be relayed and recorded more quickly and more reliably.

Consensus Fee Structure

Current consensus fees:

- No fee if:
  - Transaction less than 1000 bytes in size,
  - All outputs are 0.01 BTC or larger, and
  - Priority is large enough
- Priority = (sum of inputAge*inputValue) / (trans size)

- Otherwise fee is 0.0001 BTC per 1000 bytes

- Approx transaction size: 148 N_{inputs} + 34 $N_{outputs}$ + 10

Transaction threshold is 4 BTC-days per 1000 bytes.

Consensus Fee Structure

- Most miners enforce the consensus fee structure.
- If you don’t pay the consensus fee, your transaction will take longer to be recorded!
- Miners prioritize transactions based on fees and the priority formula.

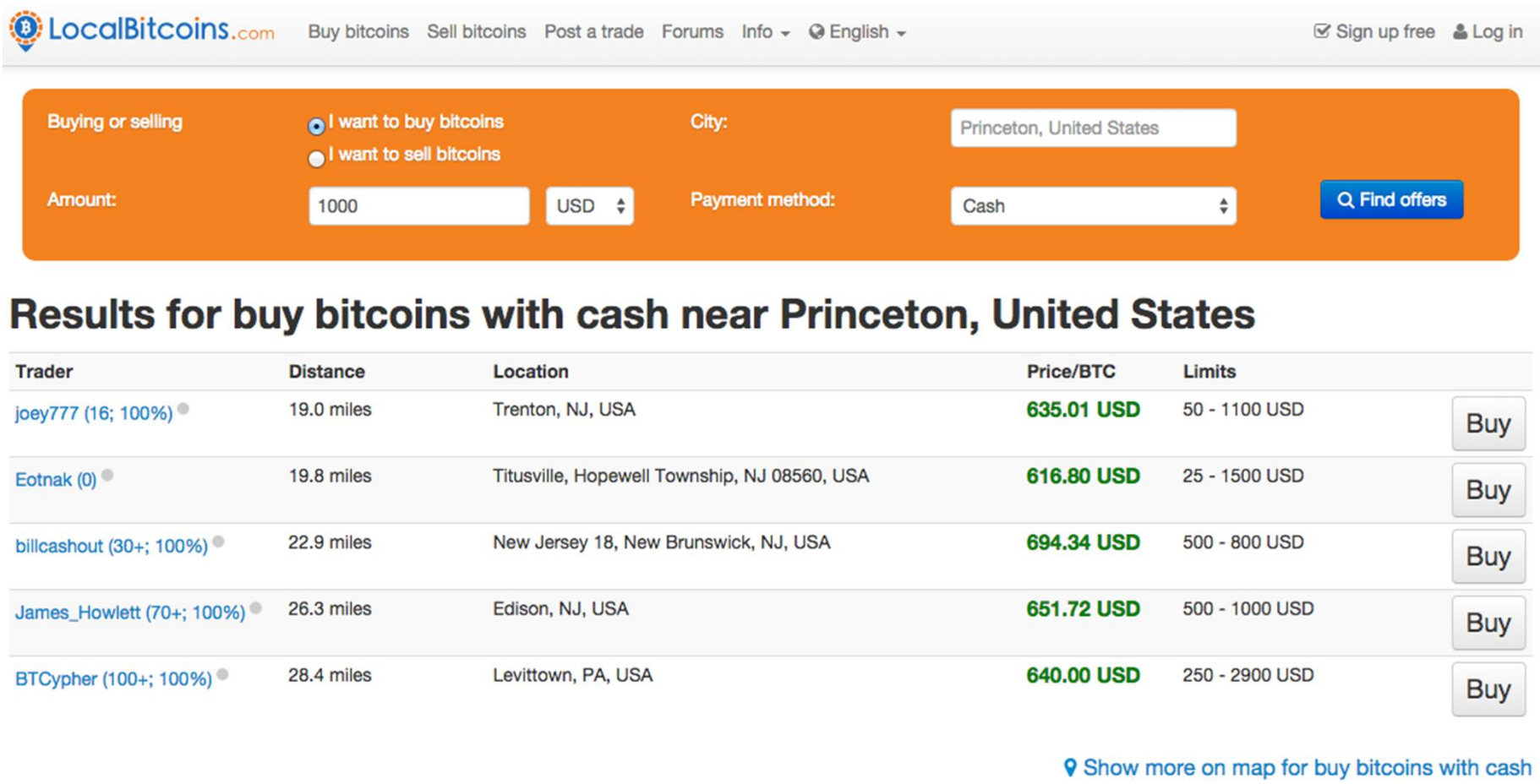
Currency Exchange Markets

Currency Exchange Markets

- By currency exchange we mean trading bitcoins against fiat currency like dollars and euros.
- In the Bitcoin world there are sites like **bitcoincharts.com** that show the exchange rate with various fiat currencies on a number of different exchanges.
- In March 2015 the volume on Bitfinex, the largest Bitcoin — USD exchange, was about 70,000 bitcoins or about 21 million dollars over a 24 hour period.
- Another option is to meet people to trade bitcoins in real life. There are sites that help you do this - localbitcoins.com.

http://bitcoincharts.com/markets

Symbol	Latest Price	30 days	Average	Volume	Low/High	Bid	Ask	24h Avg.	Volume	Low/High
▼ BitStamp USD bitstampUSD	582.54 2 min ago		620.52 -37.98 -6.12%	155,811.67 96,683,593.16 USD	570.5 658.88	581.13	582.54	585.63 -3.09 -0.53%	6,189.14 3,824,569.60 USD	574.15 596
Bitfinex USD bitfinexUSD	619.78 6 days, 5 hrs ago		632.10 -12.32 -1.95%	126,042.21 79,671,138.43 USD	593.37 665	579.31	580.49	—	0.00 0.00 USD	— —
▼ btc-e USD btceUSD	572.78 0 min ago		615.51 -42.73 -6.94%	106,578.66 65,599,931.43 USD	562 654.381	572.541	572.779	576.33 -3.55 -0.62%	3,396.32 1,957,406.31 USD	566.001 585.85
▼ itBit USD itbitUSD	581.69 just now		618.36 -36.67 -5.93%	34,726.55 21,473,457.56 USD	571 662	580.27	581.11	582.64 -0.95 -0.16%	1,607.07 936,342.67 USD	577 587.99
▲ ANX USD anxhkUSD	593.43896 29 min ago		624.73 -31.29 -5.01%	30,902.66 19,305,871.63 USD	565.166 687.21424	577.2	593.34886	587.47 5.97 1.02%	1,476.78 867,565.29 USD	565.3373 602.06006
▲ LocalBitcoins USD localbtcUSD	977.52 9 min ago		665.75 311.77 46.83%	17,221.75 11,465,390.41 USD	492.94 2529.6	1163.78	558.61	636.33 341.19 53.62%	840.60 534,896.62 USD	531.87 2500
1coin USD 1coinUSD	605.3 4 days, 6 hrs ago		625.85 -20.55 -3.28%	14,973.92 9,371,488.64 USD	601.5 664.5	605.1	605.3	—	0.00 0.00 USD	— —
▼ hitbtc USD hitbtcUSD	583.41 0 min ago		622.80 -39.39 -6.32%	14,778.51 9,203,987.87 USD	573.23 657.47	581.54	583.33	587.71 -4.30 -0.73%	459.21 289,883.25 USD	576.72 594.67
▲ CoinTrader USD cotrUSD	589.76 31 min ago		619.79 -30.03 -4.85%	1,460.39 905,136.91 USD	0.1 700	580	588.37	585.16 4.60 0.79%	76.52 44,773.46 USD	580.66 599.68
▼ Camp BX USD cbxUSD	593 1 hr, 36 min ago		633.51 -40.51 -6.40%	1,062.60 673,170.82 USD	585.14 670	595	604	606.28 -13.28 -2.19%	36.03 21,844.39 USD	585.14 626.8
▼ Ripple USD rippleUSD	583.71244672 6 min ago		621.33 -37.62 -6.05%	567.36 352,513.96 USD	574.98 655.99	582.03	585.71244671	584.69 -0.97 -0.17%	18.40 10,757.11 USD	575.6906721 590.9794998
▲ Kraken USD krakenUSD	586.5 18 min ago		625.75 -39.25 -6.27%	169.82 106,263.87 USD	574.57864 658.87046	586.5	597.75871	583.67 2.83 0.49%	1.37 800.09 USD	574.57864 591.90124
▼ bitKonan USD bitkonanUSD	581 2 hrs, 48 min ago		624.21 -43.21 -6.92%	99.32 61,997.57 USD	551 668	581.08	615	605.10 -24.10 -3.98%	2.43 1,467.97 USD	581 610
▲ The Rock Trading Company USD rockUSD	581 0 min ago		613.09 -32.09 -5.23%	77.86 47,734.81 USD	575 699.99	587.24	604.91	578.77 2.23 0.39%	2.15 1,244.36 USD	575 581
▼ Justcoin USD justUSD	579.92 16 hrs ago		624.54 -44.62 -7.14%	59.56 37,197.01 USD	578.113 700	614.93	631.21	589.41 -9.49 -1.61%	0.30 175.70 USD	578.197 599.999
▲ BitBay USD bitbayUSD	586.57 4 hrs, 57 min ago		609.30 -22.73 -3.73%	58.04 35,361.52 USD	547 631.12	586.44	588.17	586.45 0.12 0.02%	1.17 688.35 USD	583.98 586.57
▲ Vircurex USD vcxUSD	620.00124 8 hrs, 57 min ago		620.23 -0.23 -0.04%	3.76 2,329.85 USD	590 710	621	648	601.61 18.39 3.06%	0.05 30.40 USD	590 620.00124


LocalBitcoins.com
Buy bitcoins
Sell bitcoins
Post a trade
Forums
Info
English
Sign up free
Log in
Buying or selling
I want to buy bitcoins
I want to sell bitcoins
City:
Princeton, United States
Amount:
1000
USD
Payment method:
Cash
Find offers
Results for buy bitcoins with cash near Princeton, United States
Trader
Distance
Location
Price/BTC
Limits
joey777 (16; 100%)
19.0 miles
Trenton, NJ, USA
635.01 USD
50 - 1100 USD
Buy
Eotnak (0)
19.8 miles
Titusville, Hopewell Township, NJ 08560, USA
616.80 USD
25 - 1500 USD
Buy
billcashout (30+; 100%)
22.9 miles
New Jersey 18, New Brunswick, NJ, USA
694.34 USD
500 - 800 USD
Buy
James_Howlett (70+; 100%)
26.3 miles
Edison, NJ, USA
651.72 USD
500 - 1000 USD
Buy
BTCypher (100+; 100%)
28.4 miles
Levittown, PA, USA
640.00 USD
250 - 2900 USD
Buy
Show more on map for buy bitcoins with cash

Basic Market Dynamics

- Market matches buyer and seller
- Large, liquid market reaches a consensus price
- Price set by supply (of BTC) and demand (for BTC)

Supply of Bitcoins

- Supply = coins in circulation (+ demand deposits?)
- Coins in circulation: fixed number, currently ~13.1 million
- When to include demand deposits?
  - When they can actually be sold in the market.

Supply of Bitcoins

- You might also include demand deposits of bitcoins.

- That is, if someone has put money into their account in a Bitcoin exchange, and the exchange doesn't maintain a full reserve to meet every single deposit, then there will be demand deposits at that exchange that are larger than the number of coins that the exchange is holding.

Demand for Bitcoins

- BTC demanded to mediate fiat-currency transactions
  - Alice buys BTC for $
  - Alice sends BTC to Bob
  - Bob sells BTC for $

BTC “out of circulation” during this time

- BTC demanded as an investment
  - If the market thinks demand will go up in future

Demand for Bitcoins

Simple model of transaction-demand

T = Total transaction value mediated via BTC ($ / sec)
D = Duration that BTC is needed by a transaction (sec)
S = Supply of BTC (not including BTC held as long-term investments)

$\frac{S}{D}$ Bitcoins become available per second

$\frac{T}{P}$ Bitcoins needed per second

Equilibrium:

$$P = \frac{TD}{S}$$

P is the price of Bitcoin, measured in dollars per bitcoin.